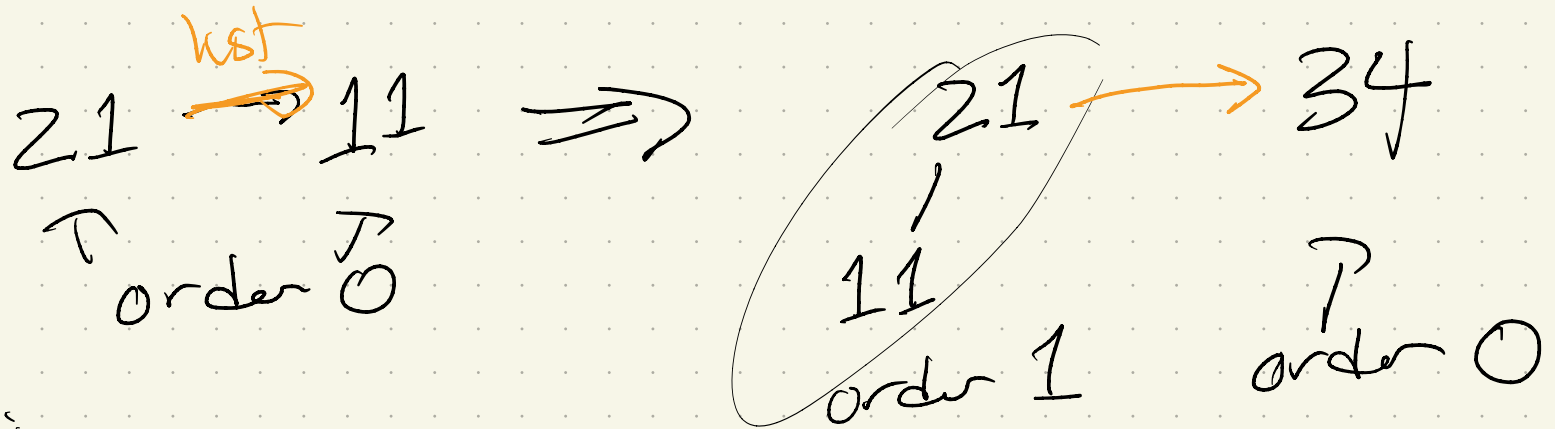


Transitions: DSR Algorithms

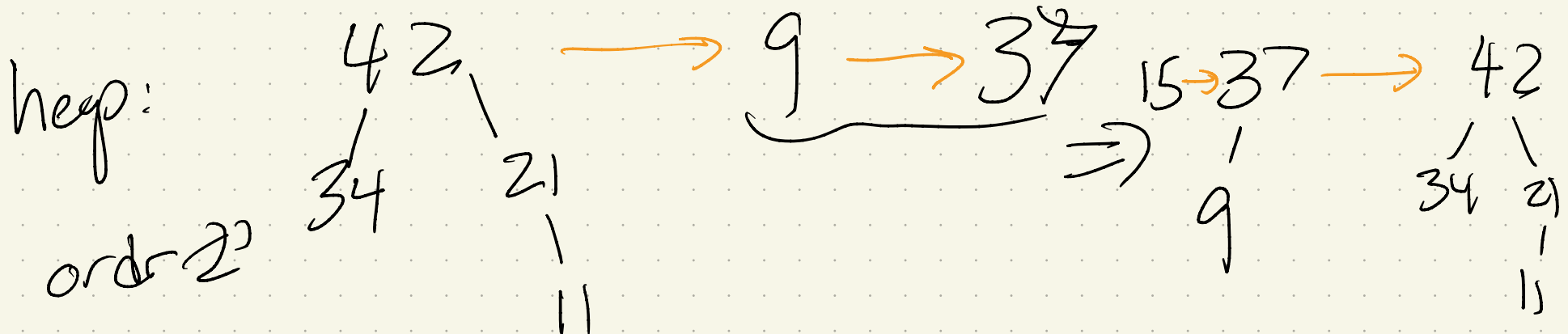
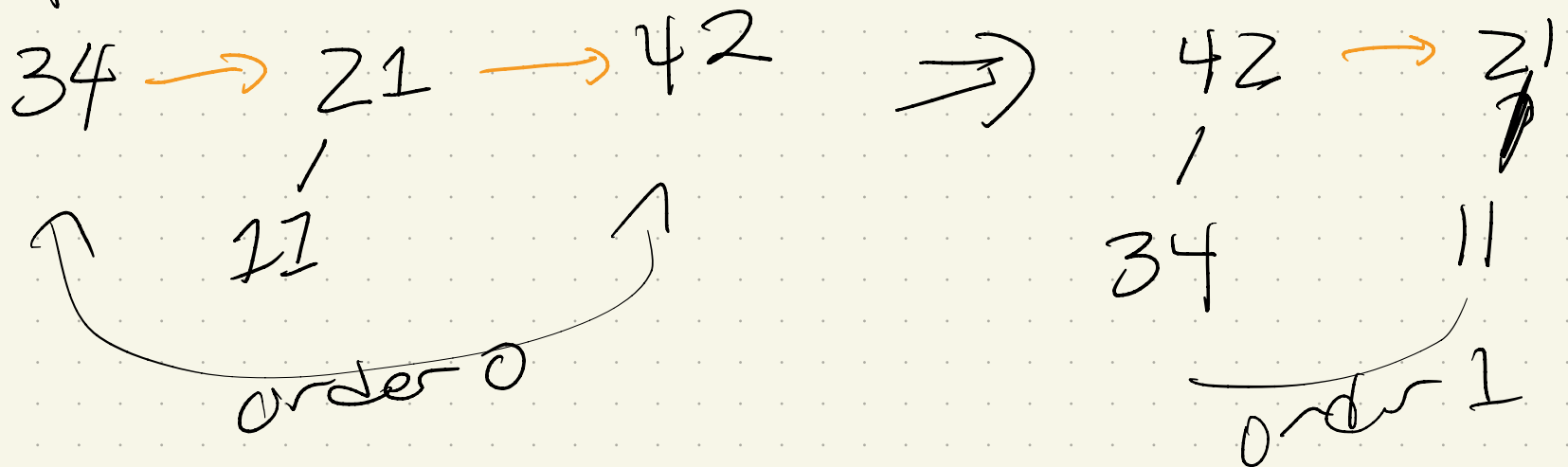
Hashing

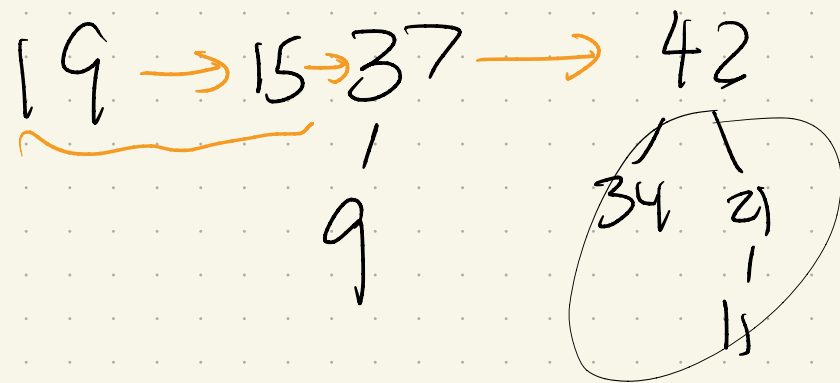


Worksheet: 21, 11, 34, 42, 9, 37, 15

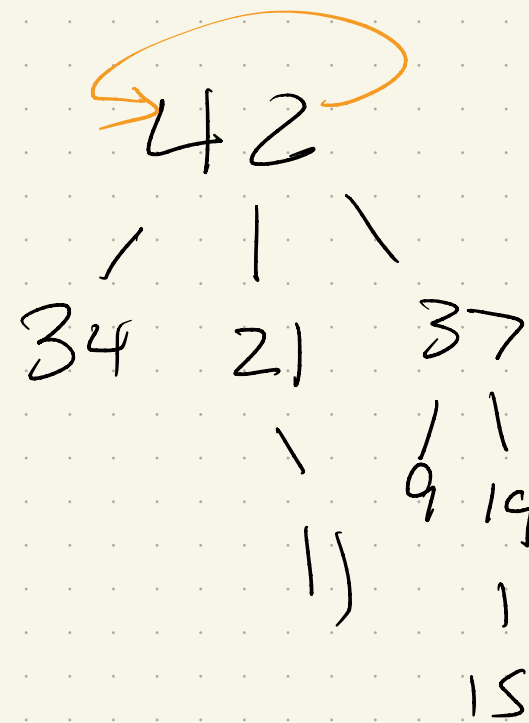
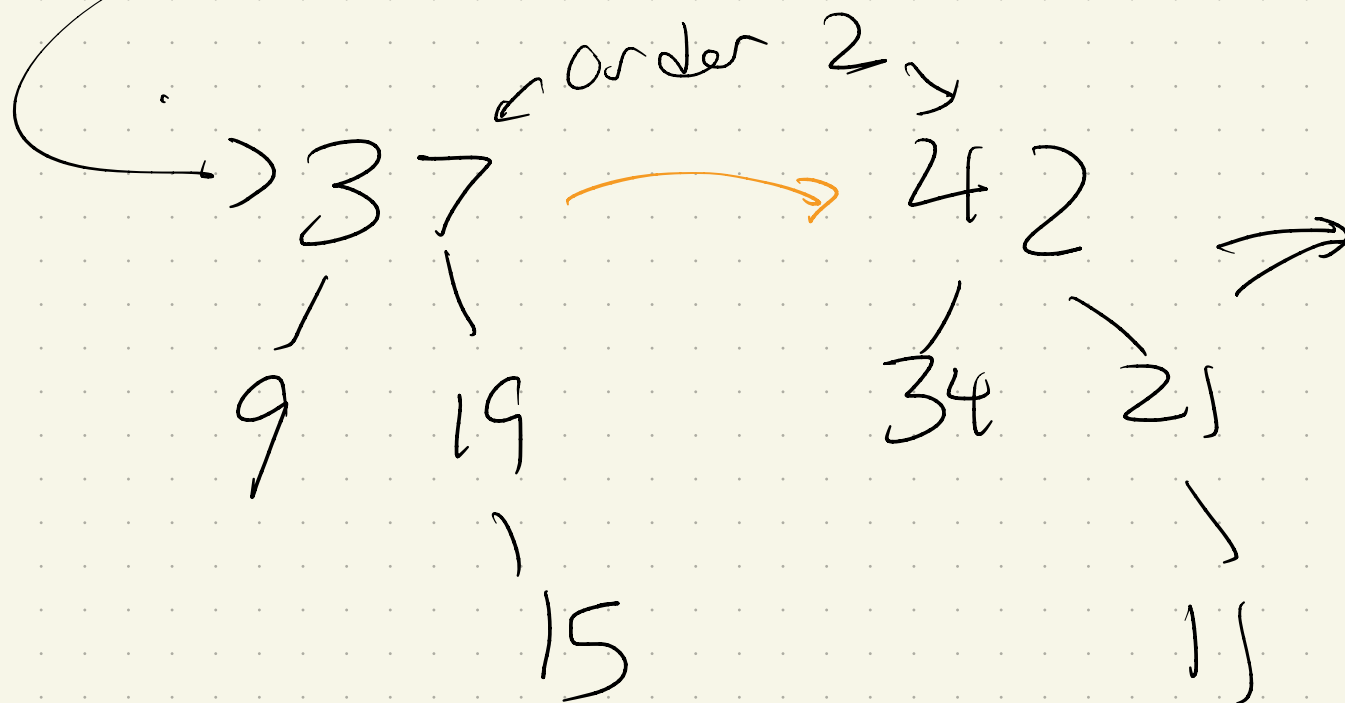
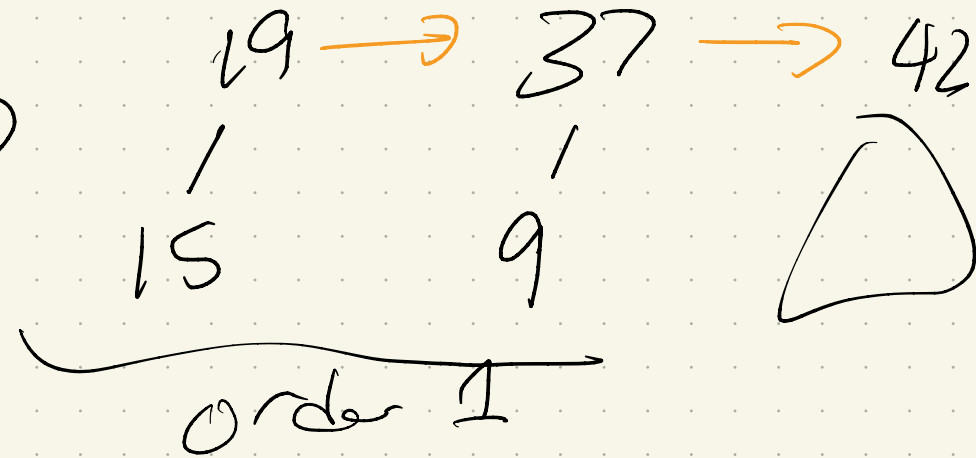


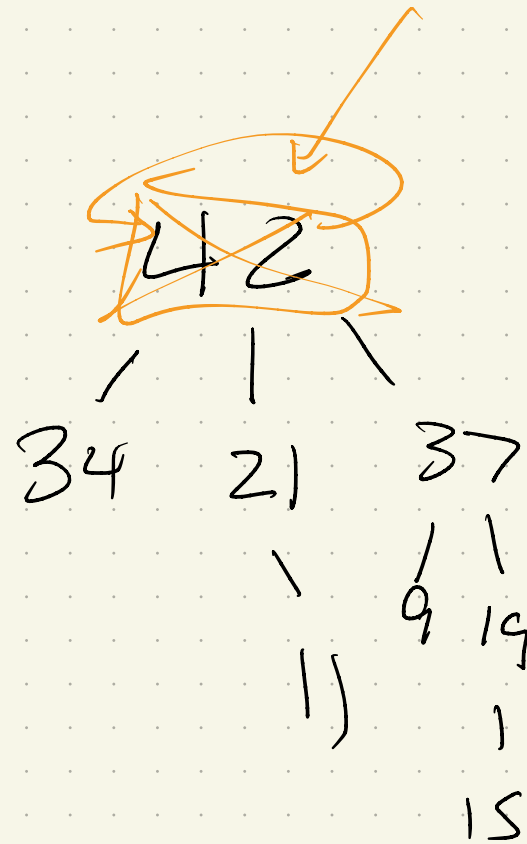
Bin heap:



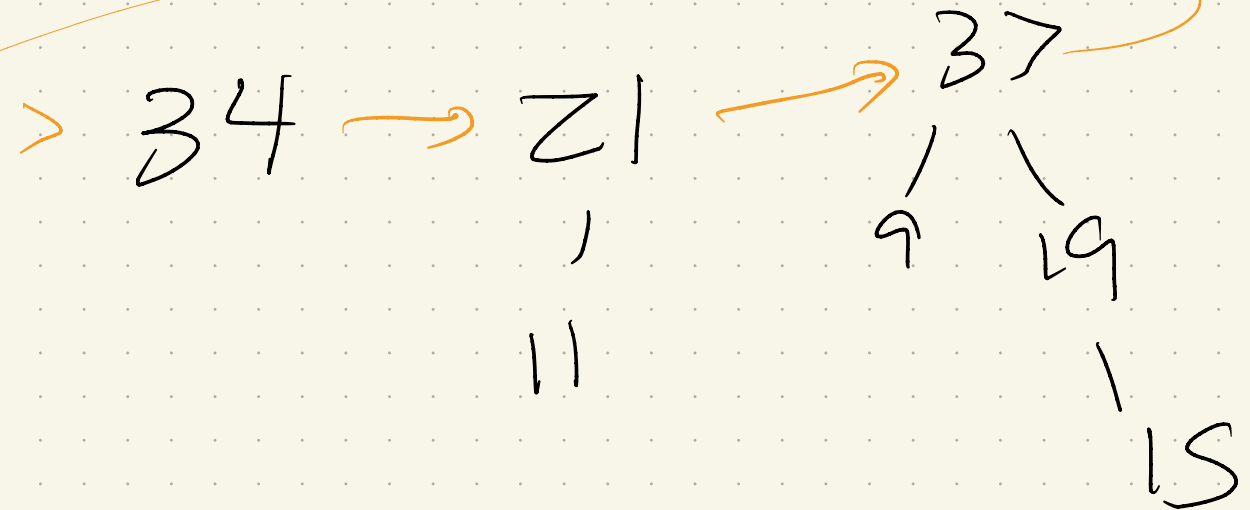
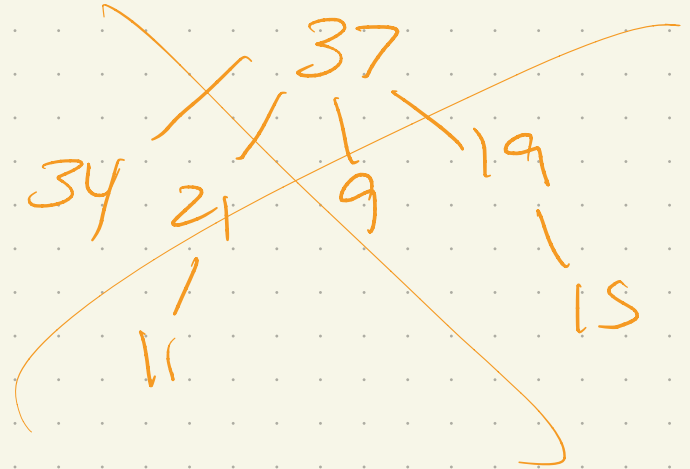


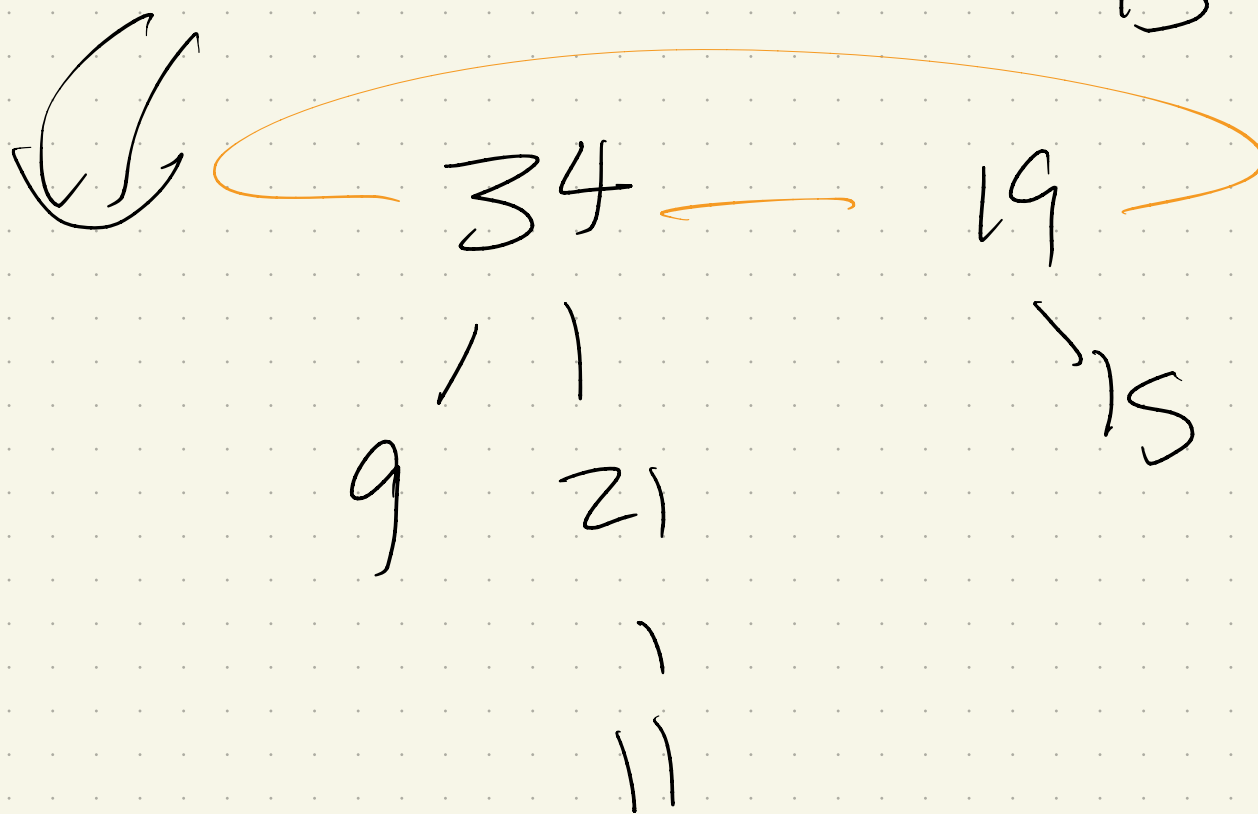
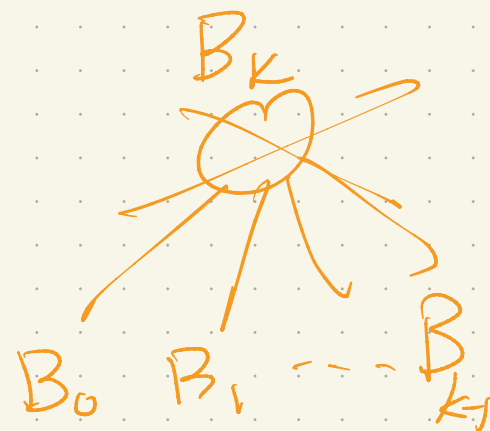
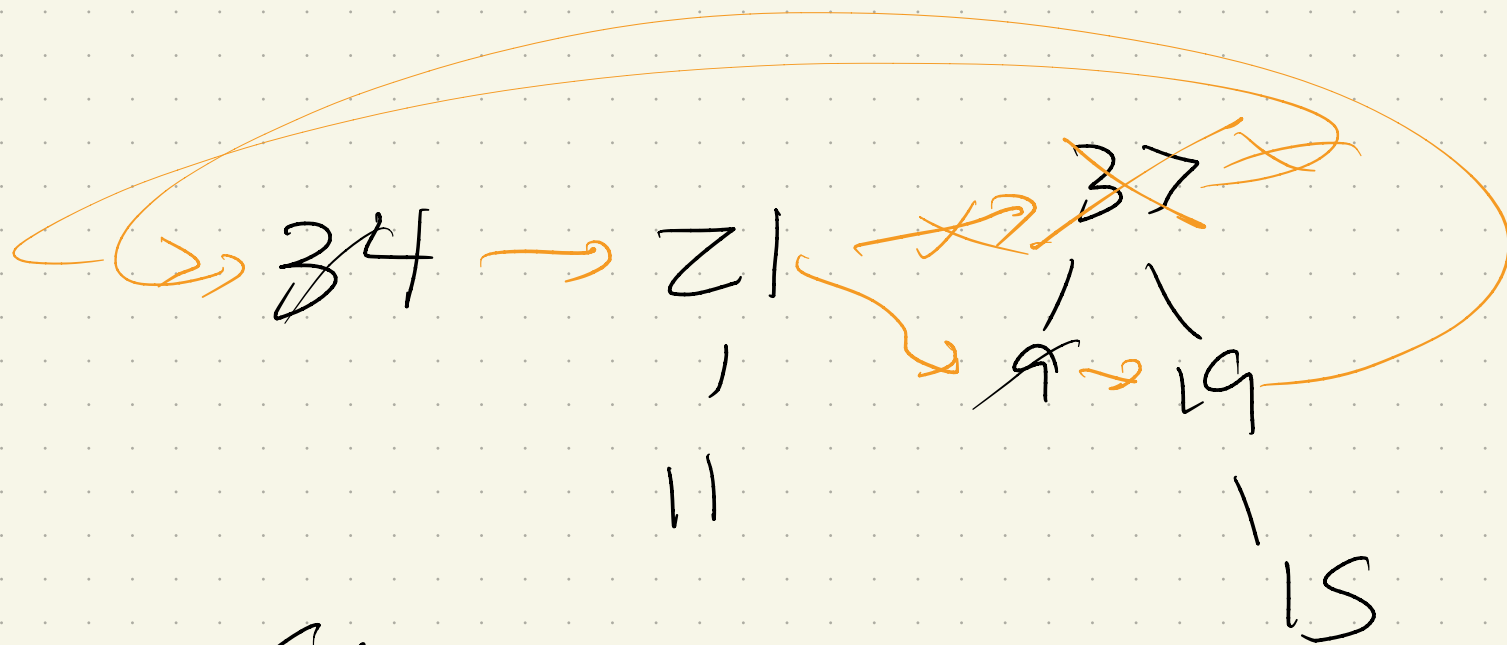
⇒





What does
remove Max do?





Show B_k has 2^k nodes:

Base case: B_0 $2^0 = 1$ ✓

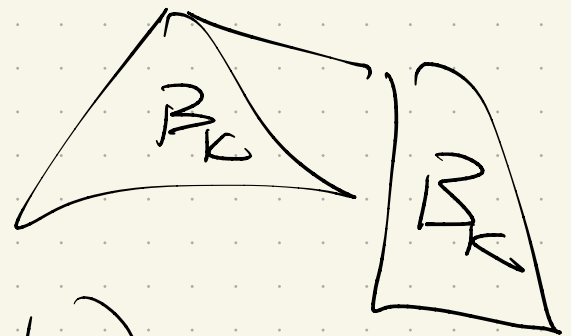
IH: Supps B_k has 2^k nodes

IS: Consider B_{k+1} :
Constructed out of 2 trees
of order k ?

So, # nodes

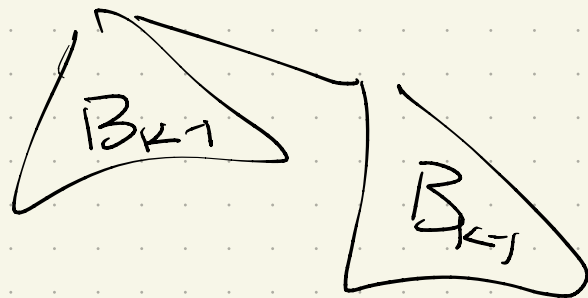
$$= 2^k + 2^k \text{ (by IH)}$$

$$= 2 \cdot 2^k = 2^{k+1}$$



B_{k-1} has 2^{k-1}

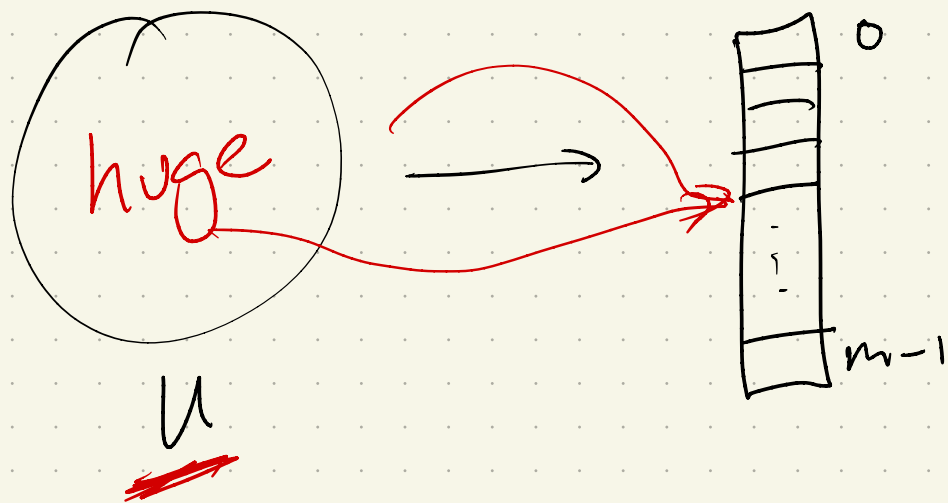
B_k :



$$\begin{aligned}\# \text{ nodes} &= 2^{k-1} + 2^{k-1} \text{ by IH} \\ &= 2 \cdot 2^{k-1} = 2^k \checkmark\end{aligned}$$

Hashing
function

$$h: \{0, \dots, U-1\} \rightarrow \{0, \dots, m-1\}$$



Problem:

U is huge,
but I need
to store
somehow

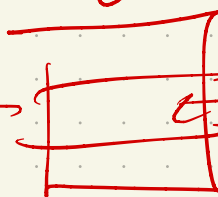
Assumption: U is huge

↳ but we won't use all of it!

Example from intro programming: dict in Python

$h(\text{"Star Wars"}) = \text{"George Lucas"}$

↑
all movies

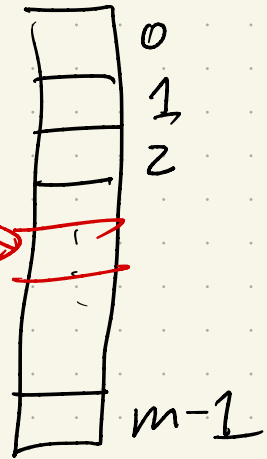


↑
star wars,
george
lucas

Natural idea: totally uniform

$$\Pr_{h \in \mathcal{H}} \{h(x) = t\} = \frac{1}{m}$$

Table:



for some family $\mathcal{H}$ of functions

Note: Not avoiding collisions!

But: how to build such functions?

Goal: $h(x)$ should be

- fast: $O(1)$ to compute
- minimize collisions

So, how to do this?

- ① Make the key a #
- ② Compress # to $[0, \dots, N-1]$
- ③ Handle collisions

First idea

For something like ASCII,
can break into pieces & treat
as bits!

Then, "chop":

Erin

69 + 114 + 105 + 110 = #

Problem: do we really expect to avoid collisions?

$h(\text{"Tom Marvolo Riddle"})$

temp 01
= temp 10
Listen = silent

$= h(\text{"I am Lord Voldemort"})$

↳ different orders

English is not very random.
Long enough string has predictable patterns.

Polynomial Hash Codes

Split data to 32-bit pieces: $X = (x_0, \dots, x_{k-1})$

Pick $a \neq 1$.

$$\text{Let } p(x) = x_0 a^{k-1} + \underline{x_1 a^{k-2}} + \dots + x_{k-2} a + x_{k-1}$$

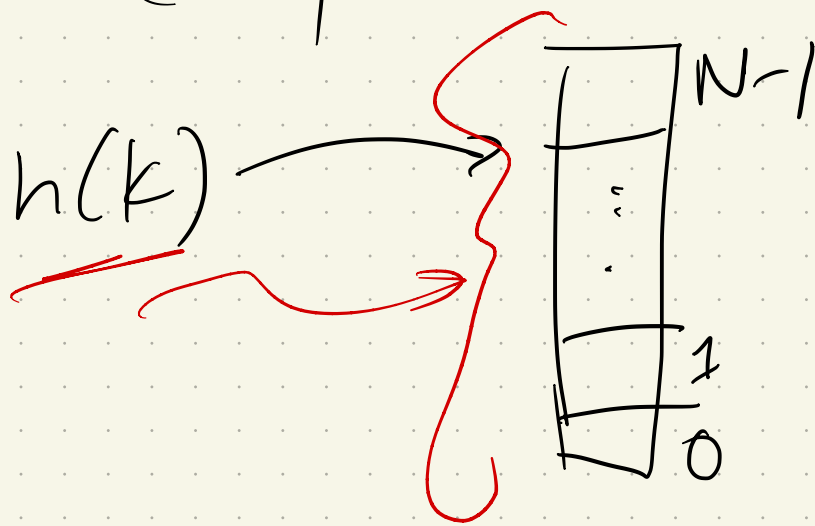
Ex: Erin (or 69, 105, 114, 110) and $a = 37$:

$$p(x) = 69 \cdot 37^3 + 105 \cdot 37^2 + 114 \cdot 37 + 110 = \#$$

Why?

- relatively fast
- helps avoid (some) collisions

Next: Compress:



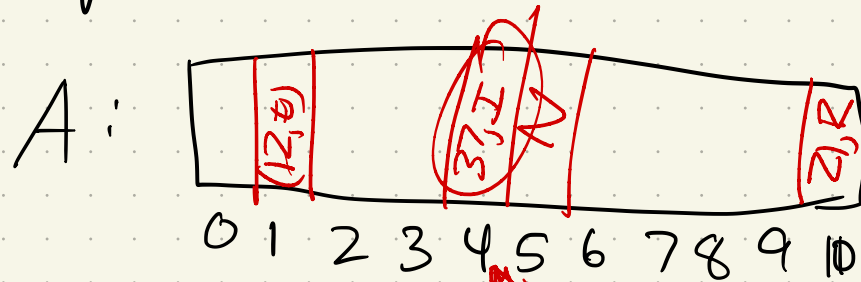
Idea: Take $h(k) \bmod N$

Recall: $3 \bmod 10 = 3$

$80 \bmod 10 = 0$

$14 \bmod 10 = 4$

Example: $h(k) = k \bmod 11$



Insert ^{key} ^{data}

(12, E)

(21, R) $\leftarrow = 10$

(37, I)

(16, N)

(26, C)

(5, H)

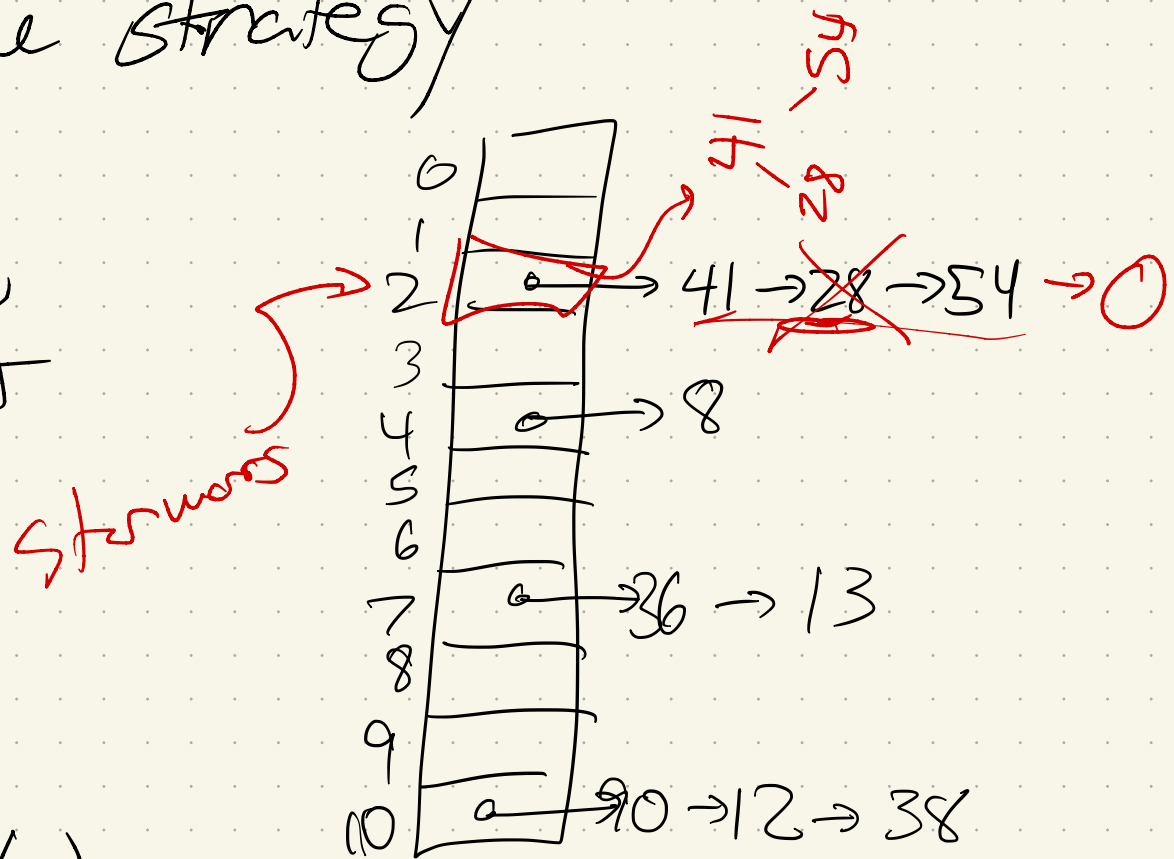
collision!

yes!

Note:
Get
new
recording!

Chaining: Simple strategy

If $h(x) = h(y)$,
store both at
this spot.



Let $l(x)$ = length
of list at $h(x)$

Time: $\Theta(1 + l(x))$

How to analyze $l(x)$?

probability

time to do linear
search is
list

Note: Practically, don't need a list:

↳ list
vector

AVL tree: $O(1 + \log_2 n)$

usually: just use list, & usually
expect $l(x) = O(1)$

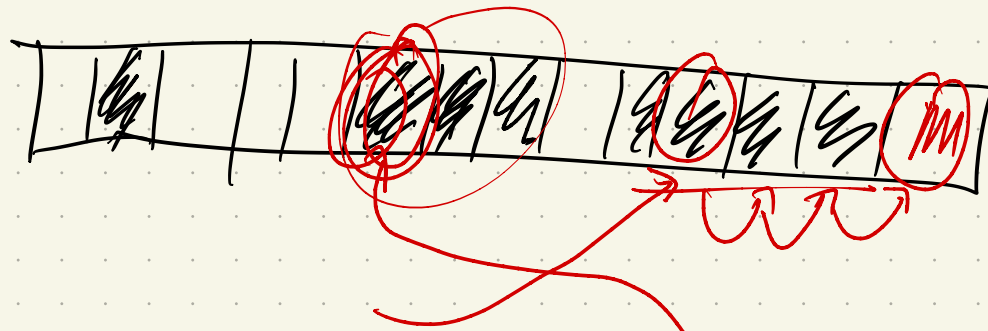
Also: Not saying $O(1)$ time expected
in worst case! ←

That is $O(1 + \underbrace{\max_x l(x)}_{\text{not } O(1)})$

provably, more like $\frac{\log_2 n}{\log_2 \log_2 n}$

Another idea:

Linear probing: if we hash to a "full" spot, just walk down to open one.



Issues:

• lookup $\rightarrow n$ ("store-wers")
 $\rightarrow$ might take $O(n)$ time

- remove?

$\rightarrow$ need to "mark", but
can't actually delete

Example: $h(k) = k \bmod 11$

A:

12E				37I	16N	26C	5H		21R	
0	1	2	3	4	5	6	7	8	9	10

cluster

Insert:

(12, E)

(21, R) $\rightarrow 21 \bmod 11 = 10$

(37, I) $\rightarrow 37 \bmod 11 = 4$

(16, N) $\rightarrow 5$

(26, C) $\rightarrow 26 \bmod 11 = 4$

(5, H)

query $\rightarrow 16$

26, C

query (4, X)

walk
array
until
empty
spot
 $\rightarrow$ report
not in

Primary clusters

$\rightarrow$ Then, remove (16, N)?

Choose a, b relatively prime

$$\hookrightarrow h(k) = ak + b \bmod N$$

works!

Load Factors

Whatever method you use, usually it starts to do badly if n gets close to N :

$\swarrow$ size of array

$$\text{Want } \frac{n}{N} < 0.5$$

$\nwarrow$ # of data insertions

Re hashing:

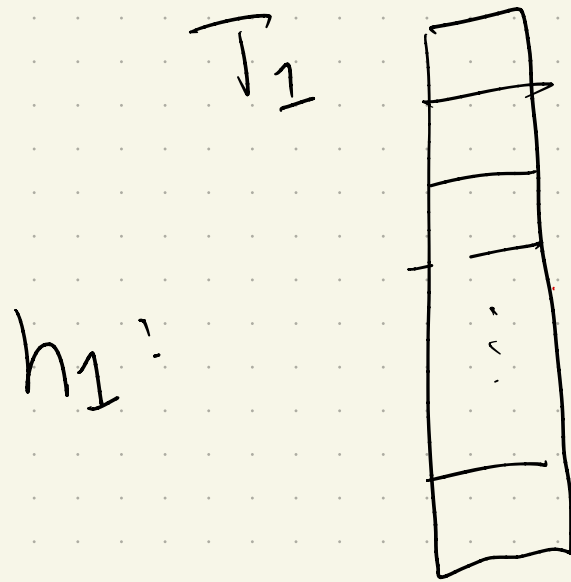
When more than half full, most implementations double the array size

(Still: not too bad:

think vectors + our
amortized analysis.)

Cuckoo Hashing

Two hash tables
2 functions:



• Maintain: x is in $h_1(x)$ or $h_2(x)$

To search:

Insert: Check $h_1(x)$:

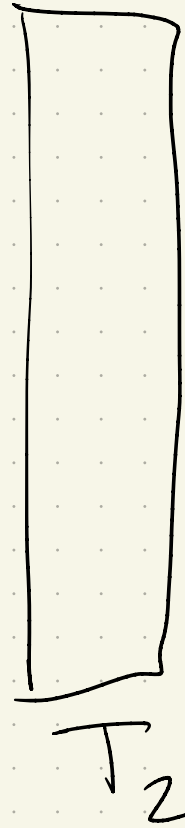
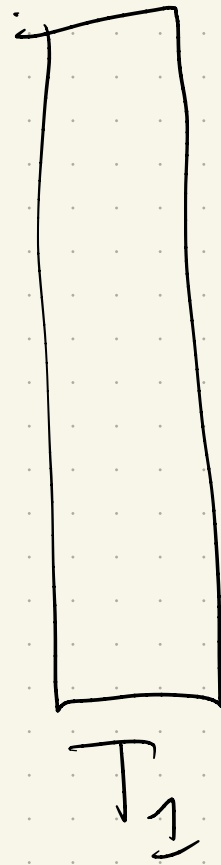
If empty:

Otherwise: evict!

Cycle $x \rightarrow y \rightarrow z \dots$

If runs too long:

Analysis: Cuckoo Graph



When are
we stuck?

So: "random graph" $\Rightarrow$ no cycles,
& short paths.

In practice:

Cuckoo Hashing

Rasmus Pagh* and Flemming Friche Rodler

BRICS**

Department of Computer Science

University of Aarhus, Denmark

{pagh,ffr}@brics.dk

in ESA, 2001

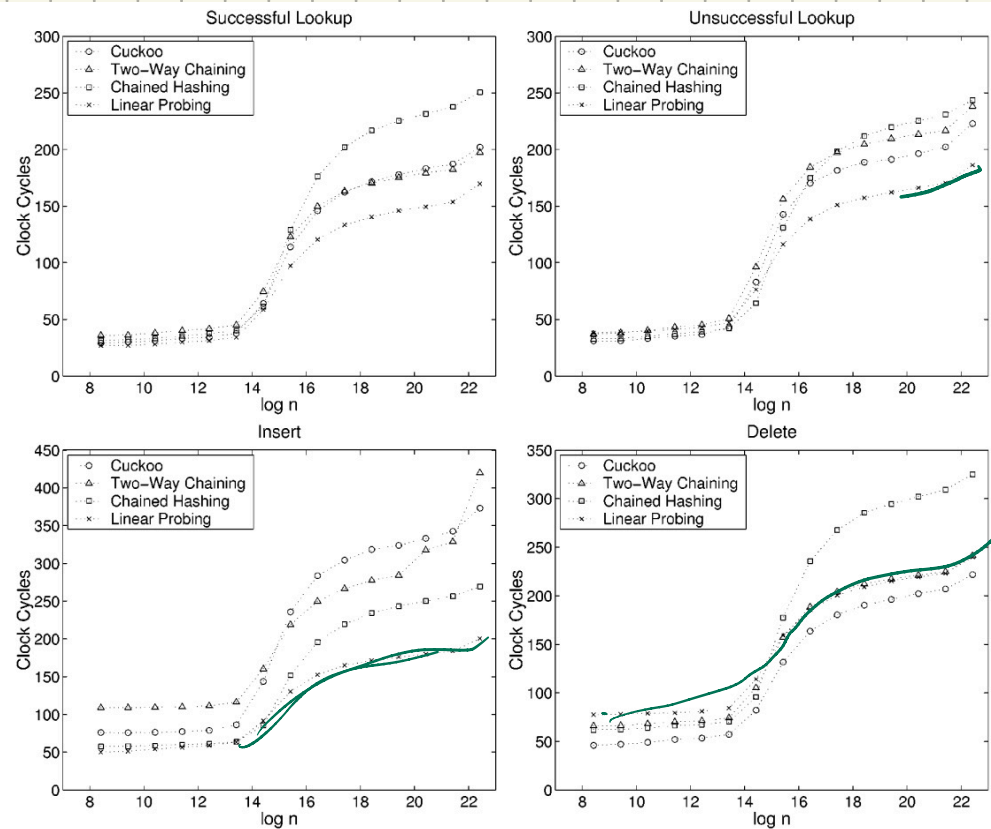


Fig. 1. The average time per operation in equilibrium for load factor 1/3.

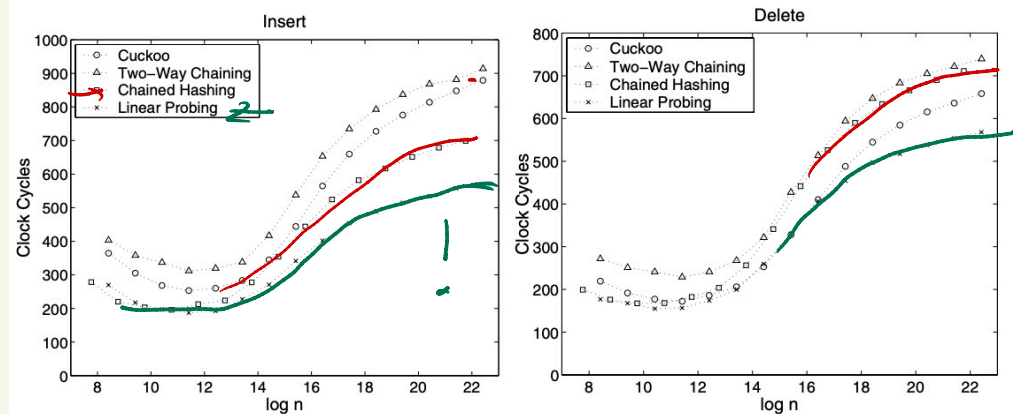


Fig. 2. The average time per insertion/deletion in a growing/shrinking dictionary for average load factor $\approx 1/3$.

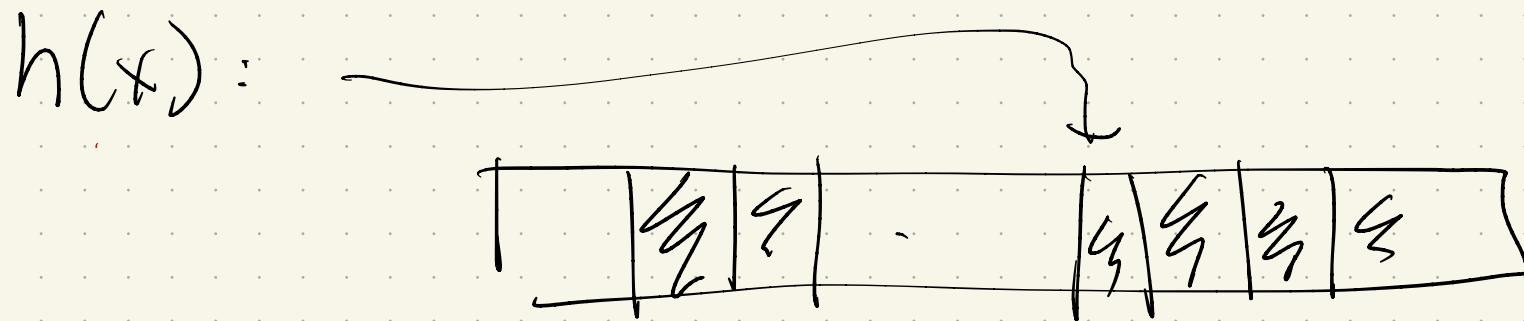
Dictionaries (hashing)

↳ tend to be $O(1)$

in practice
store entries for
later searching

Recall: Linear probing

Linear probing: If we hash to a "full" spot, just walk down to open one.



Worst case:

Expected:

If "good" hashing, each cell is full with probability $\alpha = \frac{n}{m}$

So, consider a run length R ,

$R=0$ if cell empty

$R=1$ if i full, $i+1$ empty

$R=2$ if $i, i+1$ full, $i+2$ empty

etc.

$$\Pr[R > t] = \Pr[\text{cells } i \dots i+t-1 \text{ full}]$$
$$=$$

$$E[R] = \sum_{t=1}^{\infty} \Pr(R \geq t)$$

$$= \sum_{t=1}^{\infty} \alpha^t$$

what is this?

$$\Rightarrow E[R] =$$

cells search if not present:

$$E[R+1] =$$

In reality: "primary clusters"
do happen.

But: This can help!

Why?

Caveat: Very active area!

NEWS | FREE ACCESS

Speeding Up Hash Tables

Recent hash table development raises questions about an optimal solution.

Author: Sarah Underwood [Authors Info & Claims](#)

[Communications of the ACM, Volume 69, Issue 1](#) • Pages 11 - 13 • <https://doi.org/10.1145/3772375>

Published: 02 December 2025 [Publication History](#)

Check for updates

0 5,935

All formats PDF

Hash tables are one of the oldest and simplest data structures for storing elements and supporting deletions and queries. Invented in 1953, they underly most computational systems. Yet despite their ubiquity, or perhaps because of it, computer scientists continually strive to improve their performance with a view to achieving an optimal trade-off between time and space.



2022
↓

f8v3 [cs.DS] 22 Oct 2023

Iceberg Hashing: Optimizing Many Hash-Table Criteria at Once

Michael A. Bender* Alex Conway† Martín Farach-Colton ‡
William Kuszmaul § Guido Tagliavini ¶

Abstract

Despite being one of the oldest data structures in computer science, hash tables continue to be the focus of a great deal of both theoretical and empirical research. A central reason for this is that many of the fundamental properties that one desires from a hash table are difficult to achieve simultaneously; thus many variants offering different trade-offs have been proposed.

This paper introduces Iceberg hashing, a hash table that simultaneously offers the strongest known guarantees on a large number of core properties. Iceberg hashing supports constant-time operations while improving on the state of the art for space efficiency, cache efficiency, and low failure probability. Iceberg hashing is also the first hash table to support a load factor of up to $1 - o(1)$ while being stable, meaning that the position where an element is stored only ever changes when resizes occur. In fact, in the setting where keys are $\Theta(\log n)$ bits, the space guarantees that Iceberg hashing offers, namely that it uses at most $\log \binom{U}{n} + O(n \log \log n)$ bits to store n items from a universe U , matches a lower bound by Demaine et al. that applies to any stable hash table.

Iceberg hashing introduces new general-purpose techniques for some of the most basic aspects of hash-table design. Notably, our indirection-free technique for dynamic resizing, which we call waterfall addressing, and our techniques for achieving stability and very-high probability guarantees, can be applied to any hash table that makes use of the front-yard/backyard paradigm for hash table design.